

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA



TRABAJO PRÁCTICO INTEGRADOR

MATERIA: Programación I

ALUMNOS:

- Jorge Nahuel Fernandez
- Francisco Felix Rodriguez Abalos

CARRERA: Tecnicatura Universitaria en Programación

CICLO LECTIVO: 2026

FECHA DE ENTREGA: 13-06-2026

CONTENIDO

1. Marco teórico	2
1.1 Archivos CSV y el módulo csv	2
1.2 estructuras de datos: listas y diccionarios	2
1.3 Funciones y modularización	2
1.4 Manejo de excepciones	3
1.5 Ordenamiento con list.sort()	3
2. Decisiones Técnicas y Arquitectura	4
2.1 Decisiones de diseño	4
2.2 Diagrama de flujo — Operaciones principales	5
2.3 Ejemplos de ejecución	6
3. Dificultades y conclusiones	8
3.1 Obstáculos técnicos encontrados	8
3.2 Reflexiones finales y aprendizajes	9
4. Bibliografía	9

1. MARCO TEÓRICO

1.1 ARCHIVOS CSV Y EL MÓDULO CSV

Un archivo CSV (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos están separados por comas. Es uno de los formatos más utilizados para el intercambio de datos tabulares por su simplicidad y compatibilidad.

Python incluye el módulo `csv` en su biblioteca estándar, el cual provee las clases `DictReader` y `DictWriter`. `DictReader` permite leer cada fila como un diccionario Python donde las claves corresponden a los encabezados del archivo, facilitando el acceso a los campos por nombre en lugar de por índice. `DictWriter` realiza la operación inversa: escribe diccionarios como filas en el archivo. (1)

1.2 ESTRUCTURAS DE DATOS: LISTAS Y DICCIONARIOS

En Python, una lista es una colección ordenada y mutable que permite almacenar elementos de distintos tipos. En este programa se utiliza una lista de diccionarios para representar el conjunto de países en memoria, donde cada diccionario contiene los campos `nombre`, `población`, `superficie` y `continente`.

Un diccionario es una estructura de datos que almacena pares clave-valor. Permite acceso en tiempo $O(1)$ promedio a sus elementos. En el programa se emplea también para el conteo de países por continente, inicializando las claves con los nombres de los continentes y los valores en 0. (2)

1.3 FUNCIONES Y MODULARIZACIÓN

La modularización consiste en dividir un programa en unidades más pequeñas y manejables llamadas funciones. Cada función encapsula una responsabilidad específica, lo que favorece la legibilidad, el mantenimiento y la reutilización del código.

En el programa, cada operación principal (crear, agregar, actualizar, buscar, filtrar, estadísticas) fue implementada como una función independiente, siguiendo el principio de responsabilidad única. (3)

1.4 MANEJO DE EXCEPCIONES

El manejo de excepciones permite anticipar y gestionar errores en tiempo de ejecución sin que el programa se interrumpa abruptamente. Python utiliza los bloques `try/except` para capturar excepciones específicas como `FileNotFoundError` (cuando el archivo no existe) y `ValueError` (cuando el dato ingresado no tiene el tipo o valor esperado). (4)

1.5 ORDENAMIENTO CON `LIST.SORT()`

El método `list.sort()` ordena una lista in-place (modifica la lista original). Acepta el parámetro `key`, que recibe una función que determina el criterio de comparación. El parámetro `reverse=True` permite ordenar de forma descendente. A diferencia de `sorted()`, no crea una nueva lista, lo que es más eficiente en memoria. (5)

2. DECISIONES TÉCNICAS Y ARQUITECTURA

2.1 DECISIONES DE DISEÑO

Uso de `csv.DictReader` / `DictWriter` : Se eligió `DictReader` en lugar del lector básico `csv.reader` para acceder a los campos por nombre (`fila["nombre"]`) en vez de por índice (`fila[0]`). Esto hace el código más legible y menos propenso a errores si se cambia el orden de las columnas en el CSV.

Lista de diccionarios como estructura central: Los datos se cargan desde el CSV a una lista de diccionarios en memoria antes de procesarlos. Esto permite aplicar operaciones como `sort()`, slicing y comprensiones de lista sin necesidad de releer el archivo en cada operación.

Diccionario de continentes inicializado en 0: Para garantizar que todos los continentes aparezcan en las estadísticas (incluso con 0 países), se inicializó el diccionario con los cinco continentes antes de contar. Esto evita que continentes sin países queden ausentes del resultado.

Funciones auxiliares con `key` en lugar de `lambda`: Para el ordenamiento se definieron funciones auxiliares (`obtener_nombre`, `obtener_poblacion`, `obtener_superficie`) en lugar de usar expresiones `lambda`, por ser más explícitas y fáciles de leer para el nivel de la materia.

Validación con `str.split()` e `isalpha()`: Para validar nombres que pueden contener espacios (como "Estados Unidos"), se divide la cadena con `split()` y se verifica que cada palabra sea alfabética con `isalpha()`. Esto permite nombres compuestos sin aceptar caracteres inválidos como números o símbolos.

2.2 DIAGRAMA DE FLUJO — OPERACIONES PRINCIPALES

A continuación, se describe el flujo de cada operación principal del sistema:

Operación	Flujo
Agregar país	Abrir archivo (modo 'a') → Solicitar y validar nombre, continente, población y superficie → Escribir línea al final del CSV
Actualizar datos	Abrir CSV (modo 'r') → Cargar todas las filas en memoria → Solicitar nombre → Buscar coincidencia → Validar y actualizar campos → Reescribir CSV completo (modo 'w')
Buscar país	Solicitar nombre → Abrir CSV → Recorrer filas con DictReader → Búsqueda parcial con operador 'in' → Mostrar primer resultado encontrado
Filtrar	Cargar CSV en lista → Mostrar submenú → Ordenar lista con <code>sort(key=...)</code> → Mostrar resultados
Estadísticas	Cargar CSV en lista → Contar por continente → Calcular mayor/menor población → Calcular promedios → Mostrar resultados

2.3 EJEMPLOS DE EJECUCIÓN

Menú principal

```
=====| Menú Principal | =====
1) Crear Lista
2) Agregar un País
3) Actualizar Datos
4) Buscar
5) Filtrar
6) Estadísticas
7) Salir
=====
Ingrese una opción: 
```

Opción 6 — Estadísticas

```
=====
Ingrese una opción: 6

=====
Cantidad de países por continente
=====
      América   : 2 países
      Asia      : 1 país
      Europa    : 1 país
      África    : 0 países
      Oceanía   : 0 países
=====
Estadísticas generales:
=====

País con mayor población: Brasil (213,993,437 hab.)
País con menor población: Argentina (45,376,763 hab.)
El promedio de población es: 117,079,875 hab.
El promedio de superficie es: 3,007,791 km²
=====
Presione Enter para continuar
```

Opción 4 — Buscar

```
=====| Menú Principal | =====
1) Crear Lista
2) Agregar un País
3) Actualizar Datos
4) Buscar
5) Filtrar
6) Estadísticas
7) Salir
=====
Ingrese una opción: 4
Ingrese el país: arg
Nombre: Argentina | Poblacion: 45376763 | Superficie: 2780400 | Continente: América
Presione Enter para continuar...
```

Opción 5 — Filtrar por población (ascendente)

```
=====| Menú Principal | =====
1) Crear Lista
2) Agregar un País
3) Actualizar Datos
4) Buscar
5) Filtrar
6) Estadísticas
7) Salir
=====
Ingrese una opción: 5
Filtrar
1)Por Nombre
2)Por Poblacion
3)Por Superficie
Opcion: 2
Nombre: Argentina | Poblacion: 45376763 | Superficie: 2780400 | Continente: América
Nombre: Alemania | Poblacion: 83149300 | Superficie: 357022 | Continente: Europa
Nombre: Japón | Poblacion: 125800000 | Superficie: 377975 | Continente: Asia
Nombre: Brasil | Poblacion: 213993437 | Superficie: 8515767 | Continente: América
Presione Enter para continuar...
```

Opción 5 — Filtrar por Superficie (ascendente)

```
Ingrese una opción: 5
Filtrar
1)Por Nombre
2)Por Poblacion
3)Por Superficie
Opcion: 3
1) Ascendente 2) Descendente
Opcion: 1
Nombre: Alemania | Poblacion: 83149300 | Superficie: 357022 | Continente: Europa
Nombre: Japón | Poblacion: 125800000 | Superficie: 377975 | Continente: Asia
Nombre: Argentina | Poblacion: 45376763 | Superficie: 2780400 | Continente: América
Nombre: Brasil | Poblacion: 213993437 | Superficie: 8515767 | Continente: América
```

3. DIFICULTADES Y CONCLUSIONES

3.1 OBSTÁCULOS TÉCNICOS ENCONTRADOS

Durante el desarrollo del trabajo se presentaron las siguientes dificultades técnicas:

Problema	Síntoma	Solución
Doble for sobre DictReader	Se perdía la primera fila de datos al leer el CSV	Se eliminó el bucle exterior; DictReader ya omite el encabezado automáticamente
Claves del diccionario con formato incorrecto	Algunos continentes mostraban "<15" en la salida	Se identificó que las comillas dobles dentro del f-string cerraban la cadena prematuramente; se cambió a comillas simples
Reescritura del CSV al actualizar	Al actualizar un país se perdían los demás registros	Se cargó todo el archivo en memoria con list(lector) antes de modificar, y luego se reescribió completo con DictWriter
Validación de nombres compuestos	Nombres como 'Estados Unidos' eran rechazados por isalpha()	Se aplicó split() para validar cada palabra por separado

3.2 REFLEXIONES FINALES Y APRENDIZAJES

El desarrollo de este trabajo práctico nos permitió aplicar de forma integrada los conceptos trabajados a lo largo de la materia. Podemos mencionar los principales aprendizajes:

- Modular el código en funciones independientes facilitó la detección y corrección de errores, ya que cada función tenía una responsabilidad clara y acotada.
- El manejo de archivos CSV con el módulo csv demostró ser más robusto que la lectura manual línea a línea, especialmente al trabajar con campos que podrían contener comas internas. Nos ayudó a conocer y aplicar los diferentes modos de apertura, comprendiendo para que sirven y cual elegir correctamente para cada paso.
- El uso de try/except resultó fundamental para crear una mejor experiencia del usuario, mostrándole lo que tiene que hacer en cada caso erróneo y evitando que errores esperables (como ingresar letras donde se esperan números) corten la ejecución del programa.
- La decisión de usar lista de diccionarios simplificó las operaciones de búsqueda, ordenamiento y estadísticas, ayudándonos a ordenar el código y no generar líneas extras mejorando el entendimiento de la funcionalidad de diccionarios y su aplicación.
- Completar el trabajo en grupo fomentó la revisión cruzada del código, lo que permitió detectar errores que individualmente hubieran pasado inadvertidos.

En conclusión, el proyecto nos ayudó a consolidar habilidades bases y fundamentales de programación estructurada en Python, para el trabajo con estructuras de datos, archivos y validaciones en futuros proyectos de mayor complejidad. Mejorando la lógica y el pensamiento gracias al uso de los materiales entregados por el cuerpo docente, junto a la investigación extracurricular con diferentes herramientas como foros, documentación oficial e IA.

4. BIBLIOGRAFÍA

- (1) Fuente: Python Software Foundation. (2024). csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- (2) Fuente: Python Software Foundation. (2024). Built-in Types — Mapping Types. <https://docs.python.org/3/library/stdtypes.html>
- (3) Fuente: 5 - Funciones.docx.pdf. https://tup.sied.utn.edu.ar/pluginfile.php/25771/mod_label/intro/5%20-%20Funciones.docx.pdf
- (4) Fuente: Python Software Foundation. (2024). Errors and Exceptions. <https://docs.python.org/3/tutorial/errors.html>
- (5) Fuente: Python Software Foundation. (2024). Sorting HOW TO. <https://docs.python.org/3/howto/sorting.html>

